Library Reference

A Logisim library holds a set of tools that allow you to interact with a circuit via clicking and dragging the mouse in the canvas area. Most often, a tool is intended for adding components of a particular type into a circuit; but some of the most important tools, such as the Poke Tool and the Select Tool, allow you to interact with components in other ways.

All of the tools included in Logisim's built-in libraries are documented in this reference material.

Wiring library

Splitter

Pin

Probe

Tunnel

Pull Resistor

Clock

Constant

Power/Ground

Transistor

Transmission Gate

Bit Extender

Gates library

NOT Gate

Buffer

AND/OR/NAND/NOR Gate

XOR/XNOR/Odd Parity/Even Parity Gate

Controlled Buffer/Inverter

Plexers library

Multiplexer

Demultiplexer

Decoder

Priority Encoder

Bit Selector

Arithmetic library

Adder

Subtractor

Multiplier

Divider

Negator

Comparator

Shifter

Bit Adder

Bit Finder

Memory library

D/T/J-K/S-R Flip-Flop

Register

Counter

Shift Register

Random

RAM

ROM

Input/Output library

Button

Joystick

Keyboard

LED

7-Segment Display

Hex Digit Display

LED Matrix

TTY

Base library

Poke Tool

Edit Tool

Select Tool

Wiring Tool

Text Tool

Menu Tool

Label

Splitter

Library:
Wiring
Introduced:
2.0 Beta 1 (in Base library, moved to Wiring in 2.7.0)
Appearance:

Behavior
The splitter creates a correspondence between a multi-bit value and several separate subsets of those bits. Despite its name, it can either split a multi-bit value into component parts, or it can combine component parts into a multi-bit value - or indeed it can do both at once. A more complete description of splitters is found in the `Splitters' section of the User's Guide.
Logisim treats splitters specially when propagating values within a circuit: Whereas all other components have a computed delay for purposes of simulating their behavior, values propagate through splitters (as well as wires) instantaneously.
Note: The term splitter is a non-standard term, which is unique to Logisim as far as I know. I am unaware of any standard term for such a concept; the only term I have heard used is bus ripper, but this term is unnecessarily violent for my tastes.
Pins
To distinguish the several connecting points for a splitter, we refer to the single connecting point one side as its combined end, and we refer to the multiple connecting points on the other side as its split ends.
Combined end (input/output bit width matches Bit Width In attribute)
A value holding all of the bits traveling through the splitter.
Split ends (input/output, bit width computed based on Bit x attributes)
The number of split ends is specified in the Fan Out attribute, and each split end has an index that is at least 0 and less than the Fan Out attribute. For each split end, all bits for which Bit x refers to its index travels through that split end; the order of these bits is the same as their order within the combined end.
Attributes
When the component is selected or being added, the digits '0' through '9' alter its Fan Out attribute, Alt-0 through Alt-9 alter both the Fan Out and Bit Width In attributes, and the arrow keys alter its Facing attribute.
Facing
The location of the split ends relative to the combined end.
Fan Out
The number of split ends.
Bit Width In
The bit width of the combined end.
Appearance
Supports different ways of depicting the splitter in the circuit. The Left-handed option (the default) draws a spine going left from the combined end, with a labeled line coming from the spine for each split end. The Right-handed option is the same except the spine goes right (if you're facing according to the Facing attribute). The Centered option centers the spine so it goes in roughly equal directions left and right. And the Legacy option draws diagonal lines to each split end, without labels; this option is primarily for compatibility with versions older than 2.7.0, when this was the only option for splitter appearance.

Bit x
The index of the split end to which bit x of the combined end corresponds. The split ends are indexed starting from 0 at the top (for a splitter facing east or west) or from 0 at the left/west (for a splitter facing north or south). A bit can be specified to correspond to none of the split ends. There is no way for a bit to correspond to multiple split ends.
Sometimes you can avoid twiddling each individual Bit x attribute by bringing up the pop-up menu for a splitter (usually by right-clicking or control-clicking it). The pop-up menu includes options labeled Distribute Ascending and Distribute Descending. The Distribute Ascending option distributes the bits so that each split end receives the same number of bits, starting from end 0. (If the number of split ends doesn't divide exactly into the number of bits, then the bits are distributed as evenly as possible.) Distribute Descending does the same but starts from the highest-numbered end.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Pin

Library:
Wiring
Introduced:
2.0 Beta 1 (in Base library, moved to Wiring in 2.7.0)
Appearance:

Behavior
A pin is an output or an input to a circuit, depending on the value of its Output? attribute. In drawing a pin, Logisim represents output pins using a circle or rounded rectangle, and input pins are represented using squares or rectangles. In either case, the individual bits of the value being sent or received is displayed within the component (except within printer view, when the component only says how many bits wide the pin is).
A pin is a convenient component for interacting with a circuit, and beginning Logisim users need not use them in any other way. But a user building a circuit using several subcircuits (as described in the `Subcircuits' section of the User's Guide) will use pins also to specify the interface between a circuit and a subcircuit. In particular, a circuit layout's pin components define the pins that appear on the subcircuit component when the layout is used within another circuit. In such a circuit, the values sent and received to those locations on the subcircuit component are tied to the pins within the subcircuit layout.
Pins
A pin component has only one pin, which will be an input to the component if the pin is an output pin, and it will be an output to the component if the pin is an input pin. In either case, its bit width matches the Data Bits attribute, and its location is specified by the Facing attribute.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute, the arrow keys alter its Facing attribute, and Alt with an arrow key alters its Label Location attribute.
Facing
The side of the component where its input/output pin should be.
Output?
Specifies whether the component is an output pin or an input pin. (Note that if the pin component is an input pin, then the pin that acts as its interface within the circuit will be an output, and vice versa.)
Data Bits
The number of bits for the value that the pin handles.

Three-state?
For an input pin, this configures whether the user can instruct the pin to emit unspecified (i.e., floating) values. The attribute deals with the user interface only; it does not have any effect on how the pin behaves when the circuit layout is used as a subcircuit. For an output pin, the attribute has no effect.
Pull Behavior
For an input pin, the attribute specifies how floating values should be treated when received as an input, perhaps from a circuit using the layout as a subcircuit. With "unchanged," the floating values are sent into the layout as floating values; with "pull up," they are converted into 1 values before being sent into the circuit layout; and with "pull down," they are converted into 0 values before being sent into the circuit layout.
Label
The text within the label associated with the component.
Label Location
The location of the label relative to the component.
Label Font
The font with which to render the label.
Poke Tool Behavior
Clicking an output pin has no effect, although the pin's attributes will be displayed.
Clicking an input pin will toggle the bit that is clicked. If it is a three-state pin, then the corresponding bit will rotate between the three states.
If, however, the user is viewing the state of a subcircuit as described in the `Debugging Subcircuits' of the User's Guide, then the pin's value is pinned to whatever value the subcircuit is receiving from the containing circuit. The user cannot change the value without breaking this link between the subcircuit's state and the containing circuit's state, and Logisim will prompt the user to verify that breaking this link is actually desired.
Text Tool Behavior
Allows the label associated with the component to be edited.
Back to Library Reference

Probe

Library:
Wiring
Introduced:
2.0.3 (in Base library, moved to Wiring in 2.7.0)
Appearance:

Behavior
A probe is an element that simply displays the value at a given point in a circuit. It does not itself interact with other components.
In most respects, the probe component duplicates the functionality found in a Pin component configured as an output pin. The primary difference is that if the circuit is used as a subcircuit component, then an output pin will be a part of that interface, whereas a probe is not. They also are different in that the probe does not have a Data Bits attribute to be configured: The bit width is inferred from whatever value it happens to see on its input. Graphically, they are similar but have slightly different borders: A pin has a thick, black border, whereas a probe has a thin, gray border.
Pins
A probe component has only one pin, which will acts as an input to the probe. The width that this pin accepts is adaptive: The probe will adapt to inputs of any width.
Attributes
When the component is selected or being added, the arrow keys alter its Facing attribute.
Facing
The side of the component where its input pin should be.
Label

The text within the label associated with the component.
Label Location
The location of the label relative to the component.
Label Font
The font with which to render the label.
Radix
The base (for example, binary, decimal, or hexadecimal) in which a value is displayed.
Poke Tool Behavior
None.
Text Tool Behavior
Allows the label associated with the component to be edited.
Back to Library Reference

Tunnel

Library:
Wiring
Introduced:
2.5.0 (in Base library, moved to Wiring in 2.7.0)
Appearance:

Behavior
A tunnel acts like a wire in that it binds points together, but unlike a wire the connection is not explicitly drawn. This is helpful when you need to connect points far apart in the circuit and a network of wires would make the circuit much more ugly. The below illustration illustrates how this works.

Here, all three tunnels have the same label, a, and so the three points to which the tunnels point are connected. (If one of the tunnels were labeled something else, like b, then it would be part of a different set of tunnels.) The controlled buffer at top emits a floating output since its lower input is 0. This normally leads the wire coming from the controlled buffer to be blue; but here it is dark green because the floating output combines through the tunnel with the 0 from the pin at bottom. If the control input into the buffer changes to 1, then the controlled buffer would feed 1 into the tunnel, which would combine with 0 from the pin at bottom to result in an error value; thus, we would then see red wires feeding through all three tunnels.
Pins
A tunnel has only one pin, whose bit width matches the tunnel's Data Bits attribute. This pin is neither an input nor an output — the matching tunnels are simply connected transparently.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute and the arrow keys alter its Facing attribute.
Facing
The direction toward which the tunnel points.
Data Bits
The number of bits for the tunnel.
Label
The text within the label associated with the tunnel. This tunnel is connected to all other tunnels with exactly the same label.
Label Font
The font with which to render the label.
Poke Tool Behavior
None.
Text Tool Behavior
Allows the label associated with the tunnel to be edited.
Back to Library Reference

Pull Resistor

Library:
Wiring
Introduced:
2.5.0 (in Base library, moved to Wiring in 2.7.0)
Appearance:
Shaped:

Rectangular:

Behavior
When connected to a point, this component has an effect only when the value at that point is the floating value (Z). In this case, the resistor pulls the wire to which it is connected toward the value indicated in its Pull Direction attribute.
If it is connected to a multiple-bit value, then each bit in the value that is floating is pulled in the direction specified, while the bits that are not floating are left unchanged.
Pins
The resistor has just one pin, which is an output and has a bit width that is derived from whichever component it is connected.
Attributes
When the component is selected or being added, the arrow keys alter its Facing attribute.
Facing
The direction in which the component's pin lies from component's center.
Pull Direction
Specifies the value to which a floating value should be pulled. This could be 0, 1, or the error value.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Clock

Library:
Wiring
Introduced:
2.0 Beta 13 (in Base library, moved to Wiring in 2.7.0)
Appearance:

Behavior
The clock toggles its output value on a regular schedule as long as ticks are enabled via the Simulate menu. (Ticks are disabled by default.) A "tick" is Logisim's unit of time; the speed at which ticks occur can be selected from the Simulate menu's Tick Frequency submenu.
The clock's cycle can be configured using its High Duration and Low Duration attributes.
Note that Logisim's simulation of clocks is quite unrealistic: In real circuits, multiple clocks will drift from one another and will never move in lockstep. But in Logisim, all clocks experience ticks at the same rate.
Pins
A clock has only one pin, an output with a bit width of 1, whose value will represent the current value of the clock. The location of this pin is specified in the Facing attribute. The clock's value will toggle on its schedule whenever ticks are enabled, and it will toggle whenever it is clicked using the Poke Tool.
Attributes

When the component is selected or being added, the arrow keys alter its Facing attribute.
Facing
The side of the component where its output pin should be.
High Duration
The length of time within each cycle that the clock's output should be 1.
Low Duration
The length of time within each cycle that the clock's output should be 0.
Label
The text within the label associated with the clock component.
Label Location
The location of the label relative to the component.
Label Font
The font with which to render the label.
Poke Tool Behavior
Clicking a clock component will toggle its current output value immediately.
Text Tool Behavior
Allows the label associated with the component to be edited.
Back to Library Reference

Constant

Library:
Wiring
Introduced:
2.0 Beta 1 (in Gates library, moved to Wiring in 2.7.0)
Appearance:

Behavior
Emits the value specified in its Value attribute.
Pins
There is only one pin, an output whose bit width matches the Data Bits attribute. The location of this pin is specified in the Facing attribute. The component constantly outputs on this pin whatever value specified in the Value attribute.
Attributes
When the component is selected or being added, the hexademical digits '0' through '9' and 'a' through 'f' alter its Value attribute, Alt-0 through Alt-9 alter its Data Bits attribute, and the arrow keys alter its Facing attribute.
Facing
The direction in which the pin is located relative to where the value is drawn.
Data Bits
The bit width of the value placed onto the wire.
Value
The value, written in hexademical, that is emitted by the component. The number of bits used to specify the value cannot exceed the component's bit width.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Power/Ground

Library:
Wiring

Introduced:
2.7.0
Appearance:

Behavior
Emits a single value onto a wire. For a power element, indicated by a triangle, this value will be one (or, if the Data Bits attribute is more than 1, an all-ones value). For a ground element, indicated by an arrow of three shortening parallel lines, this value will be zero (or, if the Data Bits attribute is more than 1, an all-zero value).
The same functionality can be achieved using the more versatile Constant component. The only reason to prefer ground and power is that they are standard electronic symbols.
Pins
There is only one pin, an output whose bit width matches the Data Bits attribute. The component constantly outputs the same value on this pin: for a ground component, the output is an all-zero value, and for a power component, the output is an all-one value.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute and the arrow keys alter its Facing attribute.
Facing
The direction in which the arrow will point from the location of its pin.
Data Bits
The bit width of the value placed onto the wire.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Transistor

Library:
Wiring
Introduced:
2.7.0
Appearance:

Behavior
A transistor has two inputs, called gate and source, and one output, called drain. When diagrammed, the source input and drain output are drawn connected by a plate; Logisim draws an arrowhead to indicate the direction of flow from input to output. The gate input is drawn connected to a plate that is parallel to the plate connecting source to drain. Logisim supports two types of transistors, with slightly different behaviors described below; the P-type transistor is indicated by a circle connecting the gate input to its plate, while the N-type transistor has no such circle. Depending on the value found at gate, the value at source may be transmitted to drain; or there may be no connection from source, so drain is left floating. The determination of transmitting or disconnecting depends on the type of transistor: A P-type transistor (indicated by a circle on the gate line) transmits when gate is 0, while an N-type transistor (which has no such circle) transmits when gate is 1. The behavior is summarized by the following tables.

P-type

gate

0
1
X/Z

0
0
Z
X
source
1
1
Z
X

Z
Z
Z
Z

X
X
Z
X

N-type

gate

0
1
X/Z

0
Z
0
X
source
1
Z
1
X

Z
Z
Z
Z

X
Z
X
X
Or in summarized form:
P-type

gate
drain
0
source
1
Z
X/Z
X*

N-type

gate
drain
0
Z
1
source
X/Z
X*
* If source is Z, drain is Z; otherwise drain is X.
If the Data Bits attribute is more than 1, the gate input is still a single bit, but its value is applied simultaneously to each of the source input's bits.
An N-type transistor behaves very similarly to a Controlled Buffer. The primary difference is that a transistor is meant for more basic circuit designs.
Pins (assuming component faces east, gate line top/left)
West edge (input, bit width matches Data Bits attribute)
The component's source input that will transmit to the output if triggered by the gate input.
North edge (input, bit width 1)
The component's gate input. For P-type transistors, the transistor will transmit if the gate value is 0; for N-type transistors, this will trigger the transistor if the gate value is 1.
East edge (output, bit width matches Data Bits attribute)
The component's output, which will match the source input if indicated by the gate input, or will be floating if the gate input is the negation of what indicates negation. If gate is floating or an error value, then the output will be an error value.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute and the arrow keys alter its Facing attribute.
Type
Determines whether the transistor is P-type or N-type.
Facing
The direction of the component (its output relative to its input).
Gate Location
The location of the gate input.
Data Bits
The bit width of the component's inputs and outputs.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Transmission Gate

Library:

Wiring
Introduced:
2.7.0
Appearance:

Behavior
A transmission gate has three inputs, called source, n-gate, and p-gate; and it has one output, called drain. When diagrammed, the source input and drain output are drawn connected by two plates; Logisim draws an arrowhead to indicate the direction of flow from input to output. The two gate inputs are drawn as lines connected to plates parallel to each of the plates connecting source to drain. The p-gate input's line has a circle, while the n-gate input's line does not.

p-gate

source

drain

n-gate

The transmission gate is simply the combination of two complementary transistors. Indeed, the same behavior can be achieved in Logisim by using just one transistor. However, designers sometimes prefer to use matched pairs of transistors due to electrical issues with draining voltage that is more complex than Logisim attempts to simulate.
The values at n-gate and p-gate are expected to be opposite to each other. If p-gate is 0 while n-gate is 1, then the value found at source is transmitted to drain. If p-gate is 1 while p-gate is 0, then the connection is broken, so the value at drain is left floating. In all other cases, drain receives an error output — unless source is floating, in which case drain is floating as well. This behavior is summarized by the following table.
p-gate
n-gate
drain
0
0
X*
0
1
source
1
0
Z
1
1
X*
X/Z
any
X*
any
X/Z
X*
* If source is Z, drain is Z; otherwise drain is X.
If the Data Bits attribute is more than 1, each gate input is still a single bit, but the gate values are applied simultaneously to each of the source input's bits.
Pins (assuming component faces east, gate line top/left)
West edge (input, bit width matches Data Bits attribute)

The component's source input that will transmit to the output if triggered by the p-gate and n-gate inputs.
North edge (input, bit width 1)
The component's p-gate input.
South edge (input, bit width 1)
The component's n-gate input.
East edge (output, bit width matches Data Bits attribute)
The component's output, which will match the source input if p-gate is 0 and n-gate is 1, or it will be floating if p-gate is 1 and n-gate is 0. For all other values on p-gate and n-gate, the output is an error value.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute and the arrow keys alter its Facing attribute.
Facing
The direction of the component (its output relative to its input).
Gate Location
The location of the gate input.
Data Bits
The bit width of the component's inputs and outputs.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Bit Extender

Library:
Wiring
Introduced:
2.5.0 (in Base library, moved to Wiring in 2.7.0)
Appearance:

Behavior
The bit extender transforms a value into a value of another bit width. If it's being transformed into a smaller bit width, it is simply truncated to keep the lowest-order bits. If it's being transformed into a large bit width, the lowest-order bits are the same, and you have a choice about what the additional high-order bits will be: They can all be 0, all be 1, all match the input's sign bit (its highest-order bit), or the component can have an additional one-bit input that determines the identity of these other bits.
Pins
West edge (input, bit width from Bit Width In attribute)
The multi-bit input whose value is to be transformed.
East edge (output, bit width from Bit Width Out attribute)
The computed output.
North edge (input, bit width 1)
Specifies what the additional bits in the output should be. This pin is available only when the Extension Type attribute is Input.
Attributes
When the component is selected or being added, the digits 0 through 9 alter the Bit Width In attribute and Alt-0 through Alt-9 alter its Bit Width Out attribute.
Bit Width In
The input's bit width.
Bit Width Out
The output's bit width.

Extension Type
Assuming the output bit width exceeds the input bit width, this attribute configures what the additional output bits should be. If Zero or One, the additional bits are 0 or 1 accordingly. If Sign, the additional bits are taken to match the highest-order bit in the input. And if Input, the component has a second input on its north side whose one-bit value is used for the additional bits.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

NOT Gate

Library:
Gates
Introduced:
2.0 Beta 1
Appearance:
Shaped:

Rectangular:

Behavior
The NOT Gate emits the complement of whatever input it receives. The truth table for a NOT gate is the following.

x	out
0	1
1	0

If the input is unspecified (i.e., floating), then the output will also be unspecified - unless the "Gate Output When Undefined" option is "Error for undefined inputs," in which case the output is an error. If the input is an error value, then the output will also be.
A multi-bit NOT gate will perform the above transformation bitwise on its input.
Pins (assuming component faces east)
West edge (input, bit width according to Data Bits attribute)
The component's input.
East edge (output, bit width according to Data Bits attribute)
The output, whose value is the complement of the input value.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute and the arrow keys alter its Facing attribute.
Facing
The direction of the component (its output relative to its input).
Data Bits
The bit width of the component's input and output.
Gate Size
Determines whether to draw a larger or a smaller version of the component.
Output Value
Indicates how false and true results should be translated into output values. By default, false is indicated by a low voltage (0) and true by a high voltage (1), but one or the other can be replaced by a high-impedance (floating) value instead. This allows wired-or and wired-and connections, as illustrated in the AND/OR/NAND/NOR Gate documentation.
Label

The text within the label associated with the gate.
Label Font
The font with which to render the label.
Poke Tool Behavior
None.
Text Tool Behavior
Allows the label associated with the gate to be edited.
Back to Library Reference

Buffer

Library:
Gates
Introduced:
2.0 Beta 1
Appearance:

Behavior
The buffer simply passes through to its right output whatever input it receives on the left side. The truth table for a one-bit buffer is the following.

x	out
0	0
1	1

If the input is unspecified (i.e., floating), then the output will also be unspecified - unless the "Gate Output When Undefined" option is "Error for undefined inputs," in which case the output is an error. If the input is an error value, then the output will also be.
Buffers are the most useless of the gate components provided in Logisim; its presence in the Gates library is just as much a matter of completeness (a component for each possible one-input truth table) as it is a matter of providing useful functionality. Still, it can be occasionally useful to ensure that values propagate in only one direction along a wire.
Pins (assuming component faces east)
West edge (input, bit width according to Data Bits attribute)
The input into the component.
East edge (output, bit width according to Data Bits attribute)
The output, which always matches the input into the left side.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute and the arrow keys alter its Facing attribute.
Facing
The direction of the component (its output relative to its input).
Data Bits
The bit width of the component's inputs and outputs.
Output Value
Indicates how false and true results should be translated into output values. By default, false is indicated by a low voltage (0) and true by a high voltage (1), but one or the other can be replaced by a high-impedance (floating) value instead. This allows wired-or and wired-and connections, as illustrated in the AND/OR/NAND/NOR Gate documentation.
Label
The text within the label associated with the gate.
Label Font
The font with which to render the label.
Poke Tool Behavior

None.
Text Tool Behavior
Allows the label associated with the gate to be edited.
Back to Library Reference

AND/OR/NAND/NOR Gate

Library:
Gates
Introduced:
2.0 Beta 1
Appearance:

AND
OR
NAND
NOR
Shaped:

Rectangular:
DIN 40700:
Behavior
The AND, OR, NAND, and NOT gates each compute the respective function of the inputs, and emit the result on the output.
By default, any inputs that are left unconnected are ignored — that's if the input truly has nothing attached to it, not even a wire. In this way, you can insert a 5-input gate but only attach two inputs, and it will work as a 2-input gate; this relieves you from having to worry about configuring the number of inputs every time you create a gate. (If all inputs are unconnected, the output is the error value X.) Some users, though, prefer that Logisim insist that all inputs be connected, since this is what corresponds to real-world gates. You can enable this behavior by going to the Project > Options… menu item, selecting the Simulation tab, and selecting Error for undefined inputs for Gate Output When Undefined.
The two-input truth table for the gates is the following. (The letter X represents the error value, and the letter Z represents the floating value.)
AND

0
1
X/Z
0
0
0
0
1
0
1
X
X/Z
0
X
X

OR

0
1
X/Z
0
0
1
X
1
1
1
1
X/Z
X
1
X
NAND

0
1
X/Z
0
1
1
1
1
1
0
X
X/Z
1
X
X

NOR

0
1
X/Z
0
1
0
X
1
0
0
0
X/Z
X
0
X
In short, these components work as expected as long as all inputs are either 0 or 1. If an input is neither 0 nor 1 (it is floating or it is the error value) then the component treats it as both 0 and 1: If the output would be the same both ways (as when an AND gate has one input that is definitely 0 and a questionable second input), that will be the output value; but if the output changes depending on whether it is 0 or 1, the output is the error value.
The multi-bit versions of each gate will perform its one-bit transformation bitwise on its inputs.

Pins (assuming component faces east)
West edge (inputs, bit width according to Data Bits attribute)
The inputs into the component. There will be as many of these as specified in the Number of Inputs attribute.
Note that if you are using shaped gates, the west side of OR and NOR gates will be curved. Nonetheless, the input pins are in a line. Logisim will draw short stubs illustrating this; and if you overshoot a stub, it will silently assume that you did not mean to overshoot it. In "printer view", these stubs will not be drawn unless they are connected to wires.
East edge (output, bit width according to Data Bits attribute)
The gate's output, whose value is computed based on the current inputs as described above.
Attributes
When the component is selected or being added, the digits '0' through '9' alter its Number of Inputs attribute, Alt-0 through Alt-9 alter its Data Bits attribute, and the arrow keys alter its Facing attribute.
Facing
The direction of the component (its output relative to its inputs).
Data Bits
The bit width of the component's inputs and outputs.
Gate Size
Determines whether to draw a wider or narrower version of the component. This does not affect the number of inputs, which is specified by the Number of Inputs attribute. However, if shaped gates are selected, then the gate will be drawn with wings to accommodate additional inputs beyond what the shape naturally accommodates.
Number of Inputs
Determines how many pins to have for the component on its west side.
Output Value
Indicates how false and true results should be translated into output values. By default, false is indicated by a low voltage (0) and true by a high voltage (1), but one or the other can be replaced by a high-impedance (floating) value instead. This allows wired-or and wired-and connections, as illustrated below: At left, the buffers' Output Value attribute is floating/1 and the resistor pulls to 0, giving wired-or behavior; at right, the buffers' Output Value attribute is 0/floating and the resistor pulls to 1, giving wired-and behavior.

Label
The text within the label associated with the gate.
Label Font
The font with which to render the label.
Negate x
If yes , the input is negated before it is fed into the gate. The inputs are counted top-down if the facing is east or west, and they are counted left-to-right if the facing is north or south.
Poke Tool Behavior
None.
Text Tool Behavior
Allows the label associated with the gate to be edited.
Back to Library Reference

XOR/XNOR/Odd Parity/Even Parity Gate

Library:
Gates
Introduced:
2.0 Beta 1 for XOR/Odd/Even; 2.0 Beta 6 for XNOR
Appearance:

XOR
XNOR
Odd Parity
Even Parity
Shaped:

Rectangular:
Behavior
The XOR, XNOR, Even Parity, and Odd Parity gates each compute the respective function of the inputs, and emit the result on the output.
By default, any inputs that are left unconnected are ignored — that's if the input truly has nothing attached to it, not even a wire. In this way, you can insert a 5-input gate but only attach two inputs, and it will work as a 2-input gate; this relieves you from having to worry about configuring the number of inputs every time you create a gate. (If all inputs are unconnected, the output is the error value X.) Some users, though, prefer that Logisim insist that all inputs be connected, since this is what corresponds to real-world gates. You can enable this behavior by going to the Project > Options… menu item, selecting the Simulation tab, and selecting Error for undefined inputs for Gate Output When Undefined.
The two-input truth table for the gates is the following.
x
y
XOR
XNOR
Odd
Even
0
0
0
1
0
1
0
1
1
0
1
0
1
0
1
0
1
1
0
1
0
1
As you can see, the Odd Parity gate and the XOR gate behave identically with two inputs; similarly, the even parity gate and the XNOR gate behave identically. But if there are more than two specified inputs, the XOR gate will emit 1 only when there is exactly one 1 input, whereas the Odd Parity gate will emit 1 if there are an odd number of 1 inputs. The XNOR gate will emit 1 only when there is not exactly one 1 input, while the Even Parity gate will emit 1 if there are an even number of 1 inputs. The XOR and XNOR gates include an attribute titled Multiple-Input Behavior that allow them to be configured to use the Odd Parity and Even Parity behavior.

If any of the inputs are the error value (e.g., if conflicting values are coming into the same wire) or floating, then the output will be the error value.
The multi-bit versions of each gate will perform its one-bit transformation bitwise on its inputs.
Note: Many authorities contend that the shaped XOR gate's behavior should correspond to the odd parity gate, but there is not agreement on this point. Logisim's default behavior for XOR gates is based on the IEEE 91 standard. It is also consistent with the intuitive meaning underlying the term exclusive or: A waiter asking whether you want a side dish of mashed potatoes, carrots, peas, or cole slaw will only accept one choice, not three, whatever some authorities may tell you. (I must admit, though, that I have not subjected this statement to a rigorous test.) You can configure the XOR and XNOR gates to use parity by changing its Multiple-Input Behavior attribute.
Pins (assuming component faces east)
West edge (inputs, bit width according to Data Bits attribute)
The inputs into the component. There will be as many of these as specified in the Number of Inputs attribute.
Note that if you are using shaped gates, the west side of XOR and XNOR gates will be curved. Nonetheless, the input pins are in a line. Logisim will draw short stubs illustrating this; and if you overshoot a stub, it will silently assume that you did not mean to overshoot it. In "printer view", these stubs will not be drawn unless they are connected to wires.
East edge (output, bit width according to Data Bits attribute)
The gate's output, whose value is computed based on the current inputs as described above.
Attributes
When the component is selected or being added, the digits '0' through '9' alter its Number of Inputs attribute, Alt-0 through Alt-9 alter its Data Bits attribute, and the arrow keys alter its Facing attribute.
Facing
The direction of the component (its output relative to its inputs).
Data Bits
The bit width of the component's inputs and outputs.
Gate Size
Determines whether to draw a wider or narrower version of the component. This does not affect the number of inputs, which is specified by the Number of Inputs attribute; however, if the number of inputs exceeds 3 (for a narrow component) or 5 (for a wide component), then the gate will be drawn with "wings" to be able to accommodate the number of inputs requested.
Number of Inputs
Determines how many pins to have for the component on its west side.
Output Value
Indicates how false and true results should be translated into output values. By default, false is indicated by a low voltage (0) and true by a high voltage (1), but one or the other can be replaced by a high-impedance (floating) value instead. This allows wired-or and wired-and connections, as illustrated in the AND/OR/NAND/NOR Gate documentation.
Label
The text within the label associated with the gate.
Label Font
The font with which to render the label.
Multiple-Input Behavior (XOR and XNOR only)
When three or more inputs are provided, the XOR/XNOR gate's output will either be based on whether exactly one input is 1 (the default) or an odd number of inputs are 1.
Poke Tool Behavior
None.
Text Tool Behavior
Allows the label associated with the gate to be edited.
Back to Library Reference

Controlled Buffer/Inverter

Library:
Gates
Introduced:
2.0 Beta 1
Appearance:

Behavior
The controlled buffer and inverter, often called three-state buffers/inverters, each have a one-bit "control" input pin on the south side. The value at this control pin affects how the component behaves:
When the value on this pin is 1, then the component behaves just like the respective component (a buffer or a inverter (NOT gate)).
When the value is 0 or unknown (i.e., floating), then the component's output is also floating.
When the value is an error value (such as would occur when two conflicting values are being fed into the input), then the output is an error value.
Controlled buffers can be useful when you have a wire (often called a bus) whose value should match the output of one of several components. By placing a controlled buffer between each component output and the bus, you can control whether that component's output is fed onto the bus or not.
Pins (assuming component faces east, control line right-handed)
West edge (input, bit width matches Data Bits attribute)
The component input that will be used to compute the output if the control input is 1.
South edge (input, bit width 1)
The component's control input.
East edge (output, bit width matches Data Bits attribute)
The component's output, which will be floating if the control input is 0 or floating, the error value if the control input is the error value, and will be computed based on the west-side input if the control input is 1.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute and the arrow keys alter its Facing attribute.
Facing
The direction of the component (its output relative to its input).
Data Bits
The bit width of the component's inputs and outputs.
Gate Size
(Controlled inverter only) Determines whether to draw a larger or a smaller version of the component.
Control Line Location
The location of the control line, imagining we are facing the output from the input: If the component faces east and is right-handed, the control line is to the south; but if it is left-handed, the control line is to the north.
Label
The text within the label associated with the gate.
Label Font
The font with which to render the label.
Poke Tool Behavior
None.
Text Tool Behavior
Allows the label associated with the gate to be edited.
Back to Library Reference

Multiplexer

Library:

Plexers
Introduced:
2.0 Beta 11
Appearance:

Behavior
Copies an input on the west edge onto the output on the east edge; which of the inputs to copy is specified via the current value received through the input on the south edge. I find it useful to think of a multiplexer as analogous to a railroad switch, controlled by the select input.
(Incidentally, some authorities spell this multiplexor, but multiplexer is the predominant spelling.)
Pins (assuming component faces east, select is bottom/left)
West edge, variable number (inputs, bit width matches Data Bits attribute)
Data values, one of which is to be routed to the output. Each input data value is numbered, starting with 0 on the north.
East edge (output, bit width matches Data Bits attribute)
The output value will match the input values on the west edge whose number is the same as the value currently received through the select input on the south. If the select input contains any unspecified (i.e., floating) bits, then the output is completely floating.
South edge, left side indicated by gray circle (input, bit width matches Select Bits attribute)
Select input: The value of this input determines which input on the west edge to route to the output on the east edge.
South edge, right side (input, bit width 1)
Enable: When 0, the multiplexer's output consists of all floating bits, regardless of the data and select inputs.
Attributes
When the component is selected or being added, the digits '1' through '4' alter its Select Bits attribute, Alt-0 through Alt-9 alter its Data Bits attribute, and the arrow keys alter its Facing attribute.
Facing
The direction of the component (its output relative to its input).
Select Location
The location of the select and enable lines relative to the component.
Select Bits
The bit width of the component's select input on its south edge. The number of inputs to the multiplexer will be 2selectBits.
Data Bits
The bit width of the data being routed through the multiplexer.
Disabled Output
Specifies what each bit of the output should be when the component is disabled (i.e., when the enable pin is 0). Options include zero and floating; in the latter case, the output is effectively disconnected from any other ports.
Include Enable?
The component has an enable input when this attribute is yes. The attribute is primarily for supporting circuits built using older versions of Logisim that did not provide an enable input.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Demultiplexer

Library:
Plexers
Introduced:

2.0 Beta 11
Appearance:

Behavior
Copies the input on the west edge onto exactly one of the outputs on the east edge; which of these outputs is specified via the current value received through the input on the south edge. I find it useful to think of a demultiplexer as analogous to a railroad switch, controlled by the select input. (Incidentally, some authorities spell this demultiplexor, but demultiplexer is the predominant spelling.)
Pins (assuming component faces east, select is bottom/left)
West edge (input, bit width matches Data Bits attribute)
The value to be routed to one of the outputs on the east edge.
East edge, variable number (outputs, bit width matches Data Bits attribute)
The outputs are numbered starting with 0 on the north. An output will match the west input if its number matches the value currently received through the select input on the south; otherwise, its value will be either all-zeroes or all-floating, depending on the value of the Three-State? attribute. If the select input contains any unspecified bits, then all outputs are floating.
South edge, left side (input, bit width 1)
Enable: When 0, all outputs consist of all floating bits, regardless of the data and select inputs.
South edge, right side indicated by gray circle (input, bit width matches Select Bits attribute)
Select input: The value of this input determines to which output on the east edge to route the value received on the west edge.
Attributes
When the component is selected or being added, the digits '1' through '4' alter its Select Bits attribute, Alt-0 through Alt-9 alter its Data Bits attribute, and the arrow keys alter its Facing attribute.
Facing
The direction of the component (specifying which side has the outputs).
Select Location
The location of the select and enable lines relative to the component.
Select Bits
The bit width of the component's select input on its south edge. The number of outputs for the demultiplexer will be 2selectBits.
Data Bits
The bit width of the data being routed through the demultiplexer.
Three-state?
Specifies whether the unselected outputs should be floating (Yes) or zero (No).
Disabled Output
Specifies what each bit of the outputs should be when the component is disabled (i.e., when the enable pin is 0). Options include zero and floating; in the latter case, the outputs are effectively disconnected from any other ports.
Include Enable?
The component has an enable input when this attribute is yes. The attribute is primarily for supporting circuits built using older versions of Logisim that did not provide an enable input.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Decoder

Library:
Plexers
Introduced:

2.0 Beta 11
Appearance:

Behavior
Emits 1 on exactly one output; which output is 1 depends on the current value received through the input on the south edge.
Pins (assuming component faces east, select is bottom/left)
East edge, variable number (outputs, bit width 1)
The outputs are numbered starting with 0 on the north. Each output will be 1 if its number matches the value currently received through the select input on the south; otherwise, its value will be either zero or floating, depending on the value of the Three-State? attribute. If the select input contains any unspecified bits, then all outputs are floating.
South edge, left side (input, bit width 1)
Enable: When 0, all outputs consist of all floating bits (or zeros), regardless of the select input.
South edge, right side indicated by gray circle (input, bit width matches Select Bits attribute)
Select input: The value of this input determines which of the outputs is 1.
Attributes
When the component is selected or being added, the digits '1' through '4' alter its Select Bits attribute and the arrow keys alter its Facing attribute.
Facing
The direction of the component (specifying which side has the outputs).
Select Location
The location of the select and enable lines relative to the component.
Select Bits
The bit width of the component's select input on its south edge. The number of outputs for the decoder will be 2selectBits.
Three-state?
Specifies whether the unselected outputs should be floating (Yes) or zero (No).
Disabled Output
Specifies what each bit of the outputs should be when the component is disabled (i.e., when the enable pin is 0). Options include zero and floating; in the latter case, the outputs are effectively disconnected from any other ports.
Include Enable?
The component has an enable input when this attribute is yes. The attribute is primarily for supporting circuits built using older versions of Logisim that did not provide an enable input.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Priority Encoder

Library:
Plexers
Introduced:
2.3.0
Appearance:

Behavior
The component has a number of inputs on its west edge, with the first labeled 0 and the other numbered from there. The component determines the indices of the inputs whose values are 1, and it emits the highest index. For example, if inputs 0, 2, 5, and 6 are all 1, then the priority encoder emits a value of 110. If no inputs are 1, or if the component is disabled, then the output of the priority encoder is floating.

The priority encoder is designed so that a number of encoders can be daisy-chained to accommodate additional inputs. In particular, the component includes an enable input and an enable output. Whenever the enable input is 0, the component is disabled, and the output will be all floating bits. The enable output is 1 whenever the component is enabled and none of the indexed inputs are 1. Thus, you can take two priority encoders and connect the enable output of the first to the enable input of the second: If any of the indexed inputs to the first are 1, then the second will be disabled and so its output will be all floating. But if none of the first's indexed inputs are 1, then its output will be all-floating bits, and the second priority encoder will be enabled and it will identify the highest-priority input with a 1.
An additional output of the priority encoder is 1 whenever the priority encoder is enabled and finds a 1 on one of the indexed inputs. When chaining priority encoders together, this output can be used to identify which of the encoders was triggered.
Pins (assuming component faces east)
West edge, variable number (inputs, bit width 1)
Input values, indexed from 0 at the top/west end of the edge.
East edge, upper pin (output, bit width matches Select Bits attribute)
Output: the highest index among those inputs whose value is 1 - or all floating bits if no inputs are 1 or if the component is disabled via the Enable In input.
East edge, lower pin (output, bit width 1)
Group Signal: 1 if the component is enabled and at least one indexed input has a value of 1; otherwise this output is 0.
South edge (input, bit width 1)
Enable In: if 0, the component is disabled; otherwise the component is enabled.
North edge (output, bit width 1)
Enable Out: 1 if this component is enabled and none of the indexed inputs are 1; otherwise the output is 0.
Attributes
When the component is selected or being added, the digits '1' through '4' alter its Select Bits attribute and the arrow keys alter its Facing attribute.
Facing
The direction of the component (its output relative to its input).
Select Bits
The bit width of the component's primary output. The number of indexed inputs to the priority encoder will be 2selectBits.
Disabled Output
Specifies what each bit of the output should be when the component is disabled (i.e., when the enable pin is 0). Options include zero and floating; in the latter case, the output is effectively disconnected from any other ports.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Bit Selector

Library:
Plexers
Introduced:
2.0.5
Appearance:

Behavior
Given an input of several bits, this will divide it into several equal-sized groups (starting from the lowest-order bit) and output the group selected by the select input.

For example, if we have an eight-bit input 01010101, and we are to have a three-bit output, then group 0 will be the lowest-order three bits 101, group 1 will be the next three bits, 010, and group 2 will be the next three bits 001. (Any bits beyond the top are filled in with 0.) The select input will be a two-bit number that selects which of these three groups to output; if the select input is 3, then 000 will be the output.
Pins (assuming component faces east)
West edge (input, bit width matches Data Bits attribute)
Data value from which bits should be selected for the output.
East edge (output, bit width matches Output Bits attribute)
A group of bits from the data value, as selected by the select input.
South edge (input, bit width is quotient of Data Bits and Output Bits, rounded up)
Select input: Determines which of the bit groups should be routed to the output.
Attributes
When the component is selected or being added, the digits '0' through '9' alter its Output Bits attribute, Alt-0 through Alt-9 alter its Data Bits attribute, and the arrow keys alter its Facing attribute.
Facing
The direction of the component (its output relative to its input).
Data Bits
The bit width of the component's data input.
Output Bits
The bit width of the component's output.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Adder

Library:
Arithmetic
Introduced:
2.0 Beta 11
Appearance:

Behavior
This component adds two values coming in via the west inputs and outputs the sum on the east output. The component is designed so that it can be cascaded with other adders to provide add more bits than is possible with a single adder: The carry-in input provides a one-bit value to be added into the sum also (if it is specified), and a carry-out output provides a one-bit overflow value that can be fed to another adder.
If either of the addends contains some floating bits or some error bits, then the component will perform a partial addition. That is, it will compute as many low-order bits as possible. But above the floating or error bit, the result will have floating or error bits.
Pins
West edge, north end (input, bit width matches Data Bits attribute)
One of the two values to add.
West edge, south end (input, bit width matches Data Bits attribute)
The other of the two values to add.
North edge, labeled c in (input, bit width 1)
A carry value to add into the sum. If the value is unknown (i.e., floating), then it is assumed to be 0.
East edge (output, bit width matches Data Bits attribute)
The lower dataBits bits of the sum of the two values coming in the west edge, plus the cin bit.
South edge, labeled c out (output, bit width 1)

The carry bit computed for the sum. If the values added together as unsigned values yield a result that fits into dataBits bits, then this bit will be 0; otherwise, it will be 1.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the values to be added and of the result.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Subtractor

Library:
Arithmetic
Introduced:
2.0 Beta 11
Appearance:

Behavior
This component subtracts values coming in via the west inputs (the upper minus the lower) and outputs the difference on the east output. The component is designed so that it can be cascaded with other subtractors to provide subtract more bits than is possible with a single subtractor: The borrow-in input provides a one-bit value to be borrowed out of the difference (if the borrow-in input is specified), and a borrow-out output indicates whether the component needs to borrow an upper-order bit to complete the subtraction without underflow (assuming unsigned subtraction).
Internally, the subtractor simply performs a bitwise NOT on the subtrahend, and add this to the minuend along with the NOT of the borrow-in input. (The minuend is the first operand (upper input) to the subtraction, and the subtrahend is the second (lower input). I happen to like the antiquated terms.)
If either of the operands contains some floating bits or some error bits, then the component will perform a partial subtraction. That is, it will compute as many low-order bits as possible. But above the floating or error bit, the result will have floating or error bits.
Pins
West edge, north end (input, bit width matches Data Bits attribute)
The minuend of the subtraction; that is, the number from which to subtract.
West edge, south end (input, bit width matches Data Bits attribute)
The subtrahend of the subtraction; that is, the number to subtract from the minuend.
North edge, labeled b in (input, bit width 1)
If 1, then 1 is borrowed out of the difference. If the value is unknown (i.e., floating), then it is assumed to be 0.
East edge (output, bit width matches Data Bits attribute)
The lower dataBits bits of the difference of the two values coming in the west edge, minus the bin bit.
South edge, labeled b out (output, bit width 1)
The borrow bit computed for the difference. If the values subtracted as unsigned values yield a negative value, then this bit will be 1; otherwise, it will be 0.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the values to be subtracted and of the result.
Poke Tool Behavior
None.
Text Tool Behavior

None.
Back to Library Reference

Multiplier

Library:
Arithmetic
Introduced:
2.0 Beta 20
Appearance:

Behavior
This component multiplies two values coming in via the west inputs and outputs the product on the east output. The component is designed so that it can be cascaded with other multipliers to multiply a multiplicand with more bits than is possible with a single multiplier: The carry-in input provides a multi-bit value to be added into the product (if it is specified), and a carry-out output provides the upper half of the product result, which can be fed into another multiplier.
If the multiplicand, the multiplier, or the carry-in input contain some floating bits or some error bits, then the component will perform a partial multiplication. That is, it will compute as many low-order bits as possible. But above the floating or error bit, the result will have floating or error bits. Note that if the carry-in input is completely floating, then it will be assumed to be all-zeroes.
Pins
West edge, north end (input, bit width matches Data Bits attribute)
The multiplicand (that is, the first of the two numbers to multiply).
West edge, south end (input, bit width matches Data Bits attribute)
The multiplier (that is, the second of the two numbers to multiply).
North edge, labeled c in (input, bit width matches Data Bits attribute)
A carry value to add into the product. If all bits of the value are unknown (i.e., floating), then they are assumed to be 0.
East edge (output, bit width matches Data Bits attribute)
The lower dataBits bits of the product of the two values coming in the west edge, plus the cin value.
South edge, labeled c out (output, bit width matches Data Bits attribute)
The upper dataBits bits of the product.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the values to be multiplied and of the result.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Divider

Library:
Arithmetic
Introduced:
2.0 Beta 22
Appearance:

Behavior

This component divides two values coming in via the west inputs and outputs the quotient on the east output. The component is designed so that it can be cascaded with other dividers to provide support a dividend with more bits than is possible with a single divider: The upper input provides the upper dataBits bits of the dividend (if it is specified at all), and the rem bits provide the remainder, which can be fed as the upper input into another divider.
If the divisor is 0, then no division is performed (i.e., the divisor is assumed to be 1).
The divider essentially performs unsigned division. That is, the remainder will always be between 0 and divisor-1. The quotient will always be an integer so that
quotient * divisor + remainder = dividend .
If, however, the quotient does not fit into dataBits bits, then only the lower dataBits bits will be reported. The component does not provide any method for accessing the upper dataBits bits.
If either of the operands contains some floating bits or some error bits, then the component's outputs will be either entirely floating or entirely error values.
Pins
West edge, north end (input, bit width matches Data Bits attribute)
The lower dataBits bits of the dividend (that is, the first operand for the division).
West edge, south end (input, bit width matches Data Bits attribute)
The divisor (that is, the second operand for the division)
North edge, labeled upper (input, bit width matches Data Bits attribute)
The upper dataBits bits of the dividend (that is, the first operand for the division).
East edge (output, bit width matches Data Bits attribute)
The lower dataBits bits of the quotient, as specified above.
South edge, labeled rem (output, bit width matches Data Bits attribute)
The remainder of the division. This value will always be between 0 and divisor-1.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the values to be divided and of the result.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Negator

Library:
Arithmetic
Introduced:
2.0 Beta 22
Appearance:

Behavior
Computes the two's-complement negation of the input. This negation is performed by maintaining all the lower-order bits up to the lowest-order 1, and complementing all bits above that.
If the value to be negated happens to be the least negative value, then its negation (which cannot be represented in two's-complement form), is still the least negative value.
Pins
West edge (input, bit width matches Data Bits attribute)
The value to negate.
East edge, labeled -x (output, bit width matches Data Bits attribute)
The negation of the input. If the input happens to be the least negative value representable in dataBits bits, however, then the output matches the input.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute.

Data Bits
The bit width of the component's input and output.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Comparator

Library:
Arithmetic
Introduced:
2.0 Beta 22
Appearance:

Behavior
Compares two values, either as unsigned values or as two's-complement values, depending on the Numeric Type attribute. Normally, one of the outputs will be 1, and the other two outputs will be 0. The comparison is performed starting at the most significant bits in each number and descending downward in parallel until a location is found where the two values disagree. If, however, an error value or a floating value is encountered during this descent, then all outputs will match that error or floating value.
Pins
West edge, north end (input, bit width matches Data Bits attribute)
The first of the two values to be compared.
West edge, south end (input, bit width matches Data Bits attribute)
The second of the two values to be compared.
East edge, labeled > (output, bit width 1)
1 if the first input is greater than the second input, 0 if the first input is less than or equal the second input.
East edge, labeled = (output, bit width 1)
1 if the first input equals the second input, 0 if the first input is not equal the second input.
East edge, labeled < (output, bit width 1)
1 if the first input is less than the second input, 0 if the first input is greater than or equal the second input.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the component's inputs.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Shifter

Library:
Arithmetic
Introduced:
2.3.0
Appearance:

Behavior
This component includes two inputs, data and dist, and it has one output, which is the result of shifting data by dist places. Both data and output have the same number of bits in them. The component supports the following shift types:
Logical Left: All bits in data are shifted up dist places, with the bottom dist places filled with 0's. For example, 11001011 logically shifted left twice is 00101100. (The top two ones are lost.)
Logical Right: All bits in data are shifted down dist places, with the upper dist places filled with 0's. For example, 11001011 logically shifted right twice is 00110010. (The bottom two ones are lost.)
Arithmetic Right: All bits in data are shifted down dist places, with the upper dist places filled with repetitions of whatever the uppermost bit in data. For example, 11001011 arithmetically shifted right twice is 11110010.
Rotate Left: All bits in data are shifted up dist places, with the top dist places wrapped around into the bottom. For example, 11001011 rotated left twice is 00101111.
Rotate Right: All bits in data are shifted down dist places, with the bottom dist places wrapped around into the top. For example, 11001011 rotated right twice is 11110010.
Note that if dist contains any floating or error inputs, then the output is composed entirely of error values, since there is no way to guess how far to shift the input.
Pins
West edge, north end (input, bit width matches the Data Bits attribute)
The value to be shifted.
West edge, south end (input, bit width is computed as below)
The number of bits by which to shift the data input. This input should have as many bits as is the minimum number to indicate any shift distance from 0 up to one less than Data Bits; that is, it should be the ceiling of the base-2 logarithm of Data Bits. For example, if Data Bits were 8, this input would require 3 bits; but if it were 9, it would require 4 bits.
East edge (output, bit width matches the Data Bits attribute)
The result of shifting the input value by the input distance.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the data input and of the output.
Shift Type
One of the five possible shift types as outlined above (Logical Left, Logical Right, Arithmetic Right, Rotate Left, Rotate Right).
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Bit Adder

Library:
Arithmetic
Introduced:
2.6.0
Appearance:

Behavior
The component determines how many 1 bits are in its input(s) and emits the total number of 1 bits on its output. For example, given the 8-bit input 10011101, the output would be 5, since there are five 1-bits in the input (the first, the last, and a string of three bits in the middle).
If any of the input bits are floating or error values, then the output will contain error bits in the output corresponding to the range of possible outputs depending on whether those floating/error values are counted as zeroes or ones. For instance, if the 14-bit input is 111x10110x1101, then the

output must be at least 9 (if the x's are interpreted as zeroes) and at most 11 (if they are interpreted as ones). Thus, the output will be 10EE: The upper two bits will be 1 and 0 since all integers between 9 and 11 have 1 and 0 as their top two bits, but the lower two bits are EE since integers between 9 and 11 vary within these bits.
Pins
West edge (inputs, bit width matches Data Bits attribute)
The inputs whose 1 bits are to be counted. The number of inputs is based on the Number of Inputs attribute.
East edge (output, bit width computed as described below)
The number of input bits which are 1. The bit width of the output is the minimum number of bits to store the maximum possible value (which would be the product of the Data Bits attribute and the Number of Inputs attribute).
Attributes
When the component is selected or being added, the digits '0' through '9' alter its Number of Inputs attribute and Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the input(s).
Number of Inputs
The number of input values.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Bit Finder

Library:
Arithmetic
Introduced:
2.6.1
Appearance:

Behavior
The component takes a multi-bit input and determines the the index of a bit, where the index is computed by counting from 0 as the lowest-order bit. Exactly which index it computes depends on the Type attribute, as illustrated by the examples in the below table for the 8-bit sample input 11010100.
Type
Output for 11010100
Lowest-order 1
2
Highest-order 1
7
Lowest-order 0
0
Highest-order 0
5
For the lowest-order 1, the output is 2 because if you index the bits starting from 0 for the lowest-order bit, the first 1 you will find is at index 2. (The bits at indices 0 and 1 are both 0.) For the highest-order 1, the output is 7 because the topmost 1 bit is at index 7 (again counting from the lowest-order bit as 0).
The component's output on the south edge indicates whether the desired bit was found at all. In the above examples involving the input 11010100, the south output is 1 in all cases. But if the input

were 00000000 and the component is to find the lowest-order 1, then the south output would be 0 — and the output on the east edge would be 0 as well.
If while searching for the desired value, a value that is neither 0 or 1 is found (the bit could be floating or an error value), then both outputs will consist entirely of error bits. Note that this occurs only if the problematic bit is encountered before finding the desired bit: For the input x1010100, the output would still be 2 if the lowest-order 1 is desired; but we would get error values if the component's type indicates to search for the highest-order 1 or the highest-order 0, since there is an erroneous bit in a higher-order bit than either the highest-order 0 or the highest-order 1.
Pins
West edge (input, bit width matches Data Bits attribute)
The multibit input that is to be searched for the desired bit.
East edge (output, bit width computed as described below)
The index of the desired bit, counting from 0 for the lowest-order bit. The bit width is the minimum number of bits to store the maximum possible index, which is one less than the value of the Data Bits attribute.
South edge (output, bit width 1)
1 if the desired bit is found, 0 if all input bits are the inverse of the desired bit, and the error value if a non-0, non-1 value is found before the desired bit.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the input.
Type
Indicates which bit to search for — the lowest-order 0, the highest-order 0, the lowest-order 1, or the highest-order 1.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

D/T/J-K/S-R Flip-Flop

Library:
Memory
Introduced:
2.0 Beta 1
Appearance:

Behavior
Each flip-flop stores a single bit of data, which is emitted through the Q output on the east side. Normally, the value can be controlled via the inputs to the west side. In particular, the value changes when the clock input, marked by a triangle on each flip-flop, rises from 0 to 1 (or otherwise as configured); on this rising edge, the value changes according to the table below.
D Flip-Flop
T Flip-Flop
J-K Flip-Flop
S-R Flip-Flop
D
Q
0
0
1
1
T

Q
0
Q
1
Q'
J
K
Q
0
0
Q
0
1
0
1
0
1
1
1
Q'
S
R
Q
0
0
Q
0
1
0
1
0
1
1
1
??
Another way of describing the different behavior of the flip-flops is in English text.
D Flip-Flop: When the clock triggers, the value remembered by the flip-flop becomes the value of the D input (Data) at that instant.
T Flip-Flop: When the clock triggers, the value remembered by the flip-flop either toggles or remains the same depending on whether the T input (Toggle) is 1 or 0.
J-K Flip-Flop: When the clock triggers, the value remembered by the flip-flop toggles if the J and K inputs are both 1 and the value remains the same if both are 0; if they are different, then the value becomes 1 if the J (Jump) input is 1 and 0 if the K (Kill) input is 1.
S-R Flip-Flop: When the clock triggers, the value remembered by the flip-flop remains unchanged if R and S are both 0, becomes 0 if the R input (Reset) is 1, and becomes 1 if the S input (Set) is 1. The behavior in unspecified if both inputs are 1. (In Logisim, the value in the flip-flop remains unchanged.)
By default, the clock triggers on a rising edge — that is, when the clock input changes from 0 to 1. However, the Trigger attribute allows this to change to a falling edge (when the clock input changes from 1 to 0), a high level (for the duration that the clock input is 1), or a low level (for the duration that the clock input is 0). The level-trigger options are unavailable for the T and J-K flip-flops, because a flip-flop behaves unpredictably when told to toggle for an indeterminate amount of time.
Pins
West edge, marked by triangle (input, bit width 1)

Clock input: At the instant that this input value switches from 0 to 1 (the rising edge), the value will be updated according to the other inputs on the west edge. As long as this remains 0 or 1, the other inputs on the west edge have no effect.
West edge, other labeled pin(s) (input(s), bit width 1)
These inputs control how the flip-flop's value changes during the rising edge of the clock. Their exact behavior depends on the flip-flop; the above tables summarize their behavior.
East edge, labeled Q, north end (output, bit width 1)
Outputs the value currently stored by the flip-flop.
East edge, south end (output, bit width 1)
Outputs the complement of the value currently stored by the flip-flop.
South edge, east end (input, bit width 1)
Asynchronous reset: When 0 or undefined, this input has no effect. As long as it is 1, the flip-flop's value is pinned to 0. This occurs asynchronously - that is, without regard to the current clock input value. As long as this is 1, the other inputs have no effect.
South edge, center end (input, bit width 1)
Enable: When this is 0, clock triggers are ignored. The current bit continues to appear on the output. The clock triggers are enabled when this input is 1 or undefined.
South edge, west end (input, bit width 1)
Asynchronous set: When 1 or undefined, this input has no effect. When 1, the flip-flop's value is pinned to 1. This occurs asynchronously - that is, without regard to the current clock input value. As long as this input is 1, the other inputs have no effect, except for the asynchronous reset input, which has priority.
Attributes
Trigger
Configures how the clock input is interpreted. The value rising edge indicates that the flip-flop should update its value at the instant when the clock rises from 0 to 1. The falling edge value indicates that it should update at the instant the clock falls from 1 to 0. The high level value indicates that the flip-flop should update continuously whenever the clock input is 1. And the low level value indicates that it should update continuously when the clock input is 0. Note that the latter two options are unavailable for T and J-K flip-flops.
Label
The text within the label associated with the flip-flop.
Label Font
The font with which to render the label.
Poke Tool Behavior
Clicking a flip-flop using the Poke Tool toggles the bit stored in the flip-flop, unless the asynchronous set/reset inputs currently pin the flip-flop's value.
Text Tool Behavior
Allows the label associated with the component to be edited.
Back to Library Reference

Register

Library:
Memory
Introduced:
2.0 Beta 1
Appearance:

Behavior
A register stores a single multi-bit value, which is displayed in hexadecimal within its rectangle, and is emitted on its Q output. When the clock input (indicated by a triangle on the south edge) indicates so, the value stored in the register changes to the value of the D input at that instant. Exactly when the clock input indicates for this to happen is configured via the Trigger attribute.

The reset input resets the register's value to 0 (all zeroes) asynchronously; that is, as long as reset is 1, the value is pinned to 0, regardless of the clock input.
Pins
East edge, labeled Q (output, bit width matches Data Bits attribute)
Outputs the value currently stored by the register.
West edge, labeled D (input, bit width matches Data Bits attribute)
Data input: At the instant that the clock value rises from 0 to 1, the register's value changes to the value of the D input at that instant.
West edge, labeled en (input, bit width 1)
Enable: When this is 0, clock triggers are ignored. The current value continues to appear on the output. The clock triggers are enabled when this input is 1 or undefined.
South edge, indicated with a triangle (input, bit width 1)
Clock input: At the instant that this input value rises from 0 to 1 (the rising edge), the register's value will be updated to the value of the D input.
South edge, labeled 0 (input, bit width 1)
Asynchronous reset: When 0 or undefined, this input has no effect. As long as it is 1, the register's value is pinned to 0. This occurs asynchronously - that is, without regard to the current clock input value. As long as this is 1, the other inputs have no effect.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the value stored in the register.
Trigger
Configures how the clock input is interpreted. The value rising edge indicates that the register should update its value at the instant when the clock rises from 0 to 1. The falling edge value indicates that it should update at the instant the clock falls from 1 to 0. The high level value indicates that the register should update continuously whenever the clock input is 1. And the low level value indicates that it should update continuously when the clock input is 0.
Label
The text within the label associated with the register.
Label Font
The font with which to render the label.
Poke Tool Behavior
Clicking the register brings keyboard focus to the register (indicated by a red rectangle), and typing hexadecimal digits will change the value stored in the register.
Text Tool Behavior
Allows the label associated with the component to be edited.
Back to Library Reference

Counter

Library:
Memory
Introduced:
2.3.0
Appearance:

Behavior
The counter holds a single value, whose value is emitted on the output Q. Each time the clock input (diagrammed with a triangle on the component's south edge) triggers according to its Trigger attribute, the value in the counter may update based on the two inputs on the component's west edge: The upper input is called load and the lower is called count, and they are interpreted as follows.
load
count

trigger action
0 or z
0
The counter remains unchanged.
0 or z
1 or z
The counter increments.
1
0
The counter loads the value found at the D input.
1
1 or z
The counter decrements.
The range of counting can be configured using the Maximum Value attribute. When the counter reaches this value, the next increment wraps the counter back to 0; and if it is at 0, then a decrement will wrap the counter around back to its maximum value.
In addition to the output Q, the component also includes a single-bit output carry. This is 1 whenever the counter is at its maximum and the load and count inputs indicate that the component should increment on the next step - or when the counter is at 0 and the load and count inputs indicate to decrement at the next step.
The clear input resets the counter's value to 0 (all zeroes) asynchronously; that is, as long as the clr input is 1, the value is pinned to 0, regardless of the clock input.
Pins
East edge, labeled Q (output, bit width matches Data Bits attribute)
Outputs the value currently stored by the counter.
East edge, lower pin (output, bit width 1)
Carry: When load and count indicate to increment, this output is 1 whenever the counter is at its maximum. When load and count indicate to decrement, this output is 1 whenever the counter is at 0. At all other times, this output is 0.
West edge, top pin (input, bit width 1)
Load: When this is 1 while the count input is 0, the counter will load the value found at the data input at the next clock trigger - or, if the count input happens to be 1, the counter's value will decrement.
West edge, middle pin labeled D (input, bit with matches Data Bits attribute)
Data: When the clock triggers while load is 1 and count is 0, the counter's value changes to the value found at this input.
West edge, lower pin labeled ct (input, bit width 1)
Count: When this is 1 or unconnected, the value in the counter increments whenever the clock input is triggered - or it decrements if the load input happens to also be 1.
South edge, indicated with a triangle (input, bit width 1)
Clock: At the instant that this is triggered as specified by the Trigger attribute, the counter updates as indicated by the load and count inputs.
South edge, labeled 0 (input, bit width 1)
Clear: When 0 or undefined, this input has no effect. As long as it is 1, the counter's value is asynchronously pinned to 0. This occurs asynchronously - that is, without regard to the current clock input value. As long as this is 1, the other inputs have no effect.
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the value emitted by the component.
Maximum Value
The maximum value, at which point the counter will set its carry output.
Action On Overflow
The behavior when the counter attempts to increment beyond the maximum value or decrement beyond 0. Four possible actions are supported:
Wrap around

The next value is 0 (if incrementing - the maximum value if decrementing)
Stay at value
The counter's value remains at the maximum (or 0 if decrementing)
Continue counting
The counter continues incrementing/decrementing, keeping the number of bits as provided by the Data Bits attribute
Load next value
The next value is loaded from the D input.
Trigger
Configures how the clock input is interpreted. The value rising edge indicates that the counter should update its value at the instant when the clock rises from 0 to 1. The falling edge value indicates that it should update at the instant the clock falls from 1 to 0.
Label
The text within the label associated with the component.
Label Font
The font with which to render the label.
Poke Tool Behavior
Clicking the counter brings keyboard focus to the component (indicated by a red rectangle), and typing hexadecimal digits will change the value stored in the counter.
Text Tool Behavior
Allows the label associated with the component to be edited.
Back to Library Reference

Shift Register

Library:
Memory
Introduced:
2.3.0
Appearance:

Behavior
This register consists of several stages, where each clock may lead to each stage receiving the value in the previous stage, while a new value is loaded into the first stage. The component optionally also supports parallel loads and stores to all stages' values.
The clear input resets all stages to 0 (all zeroes) asynchronously; that is, as long as the clear input is 1, all values are pinned to 0, regardless of the clock input.
Pins
* An asterisk marks pins that exist only when the Parallel Load attribute is enabled.
West edge, top pin (input, bit width 1)
Shift: When 1 or disconnected, all stages advance with the clock trigger; but if it is 0, no advance takes place. This input is ignored if the Load input is 1.
West edge, middle pin (input, bit width matches Data Bits attribute)
Data: When advancing the stages, the value found at this input is loaded into the first stage.
West edge, bottom pin marked with triangle (input, bit width 1)
Clock: At the instant that this is triggered as specified by the Trigger attribute, the component may advance the stages or load new values.
*North edge, left pin (input, bit width 1)
Load: When this 1, the values found on the other north-edge pins are loaded into all stages at the next clock trigger. When 0 or disconnected, no load occurs.
*North edge, other pins (input, bit width matches Data Bits attribute)
Data: These values are loaded into all stages when the clock is triggered while the load input is 1. The leftmost input corresponds to the youngest stage.
South edge, left pin (input, bit width 1)
Clear: When this is 1, all stages are asynchronously reset to 0, and all other inputs are ignored.

*South edge, other pins (output, bit width matches Data Bits attribute)
Output: Emits the value stored in each stage, with the youngest stage reflected on the leftmost of the pins (next to the clear input).
East edge (output, bit width matches Data Bits attribute)
Output: Emits the value stored in the final (oldest) stage.
Attributes
When the component is selected or being added, the digits '0' through '9' alter its Number of Stages attribute and Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the value stored in each stage.
Number of Stages
The number of stages included in the component.
Parallel Load
If yes, then the component includes inputs and outputs facilitating parallel access to all the stages' values.
Trigger
Configures how the clock input is interpreted. The value rising edge indicates that the register should update its value at the instant when the clock rises from 0 to 1. The falling edge value indicates that it should update at the instant the clock falls from 1 to 0.
Label
The text within the label associated with the component.
Label Font
The font with which to render the label.
Poke Tool Behavior
If the Parallel Load attribute is no, or if the Data Bits attribute is more than 4, then poking the register has no effect. Otherwise, clicking the component will bring keyboard focus to the clicked stage (indicated by a red rectangle), and typing a hexadecimal digit will change the value stored in that stage.
Text Tool Behavior
Allows the label associated with the component to be edited.
Back to Library Reference

Random

Library:
Memory
Introduced:
2.3.0
Appearance:

Behavior
This component iterates through a pseudorandom sequence of numbers, which steps forward to the following number in the sequence each time the clock is triggered while the component is enabled. Technically speaking, the algorithm used to compute the pseudorandom sequence is a linear congruential generator: Starting from a seed r0, the following number r1 is the number
r1 = (25,214,903,917 r0 + 11) mod 248
The next value r2 is computed from r1 using the same computation, and so forth. This sequence is of 48-bit numbers; the value seen from the component is the low-order bits as configured by its Data Bits attribute, after first throwing out the lower 12 bits of the current seed.
Besides the clock input, the component also includes an enable input, which leads the clock input to be ignored when enable is 0, and the reset input, which resets the component's value asynchronously to the initial seed r0.
The initial seed is user-configurable. If it is configured at 0 (which is the default), then the seed is based on the current time; when instructed to reset through the reset input, the component computes a new seed based on the new current time.

Pins
East edge, labeled Q (output, bit width matches Data Bits attribute)
Outputs the value currently stored by the component.
West edge, top pin, labeled with a triangle (input, bit width 1)
Clock: At the instant that this is triggered as specified by the Trigger attribute, the component steps to the following number in its sequence.
West edge, bottom pin (input, bit width 1)
Enable: The component is enabled when this input is disconnected or 1; but if it is 0, then the clock input is ignored.
South edge (input, bit width 1)
Reset: When this is 1, the pseudorandom sequence asynchronously resets to the initial seed. (If seed is 0, this new seed should be different from the initial seed used previously.)
Attributes
When the component is selected or being added, Alt-0 through Alt-9 alter its Data Bits attribute.
Data Bits
The bit width of the value emitted by the component.
Seed
The starting value used for the pseudorandom sequence. If this is 0 (the default), then the starting value is based on the time that the random sequence began.
Trigger
Configures how the clock input is interpreted. The value rising edge indicates that the component should update its value at the instant when the clock rises from 0 to 1. The falling edge value indicates that it should update at the instant the clock falls from 1 to 0.
Label
The text within the label associated with the component.
Label Font
The font with which to render the label.
Poke Tool Behavior
None.
Text Tool Behavior
Allows the label associated with the component to be edited.
Back to Library Reference

RAM

Library:
Memory
Introduced:
2.0 Beta 1
Appearance:

Behavior
The RAM component, easily the most complex component in Logisim's built-in libraries, stores up to 16,777,216 values (specified in the Address Bit Width attribute), each of which can include up to to 32 bits (specified in the Data Bit Width attribute). The circuit can load and store values in RAM. Also, the user can modify individual values interactively via the Poke Tool, or the user can modify the entire contents via the Menu Tool.
Current values are displayed in the component. Addresses displayed are listed in gray to the left of the display area. Inside, each value is listed using hexadecimal. The value at the currently selected address will be displayed in inverse text (white on black).
The RAM component supports three different interfaces, depending on the Data Interface attribute.
One synchronous load/store port (default)
The component includes a single port on its east side that serves for both loading and storing data. Which it performs depends on the input labeled ld: 1 (or floating) indicates to load the data at the address designated on the component's west side, and 0 indicates to store the data given on the

port. To transmit data into and out of the component, you will need to use a Controlled Buffer component, as illustrated below.

One asynchronous load/store port
This is the same as above, except that there is no clock. The value found on the data bus is stored into memory whenever the ld input is 0. If, while the ld input is 0, the address or data changes, then an additional store occurs. This option is meant to more closely approximate the interface of many available random-access memories.
Separate load and store ports
Two data ports are provided - one on the west side for storing data, and another on the east side for loading data. This option removes the necessity of dealing with the Controlled Buffer and so it is easier to use.
Pins
A on west edge (input, bit width matches Address Bit Width attribute)
Selects which of the values in memory is currently being accessed by the circuit.
D on west edge (input, bit width matches Data Bit Width attribute)
This input is present only if "separate load and store ports" is selected for the Data Interface attribute. When a store is requested (via the clock changing from 0 to 1 while sel and str are both 1 or floating), the value found at this port is stored into memory at the currently selected address.
D on east edge (input/output or output, bit width matches Data Bit Width attribute)
If sel and ld are 1 or floating, then the RAM component emits the value found at the currently selected address on this port. If there is a single load/store port, the value read from this port is stored whenever a store is requested.
str on south edge (input, bit width 1)
Store: This input is present only if "separate load and store ports" is selected for the Data Interface attribute. When it is 1 or floating, a clock pulse will result in storing the data found on the west edge into memory (provided the sel input is also 1 or floating).
sel on south edge (input, bit width 1)
Chip select: This input enables or disables the entire RAM module, based on whether the value is 1/floating or 0. The input is meant primarily for situations where you have multiple RAM units, only one of which would be enabled at any time.
triangle on south edge (input, bit width 1)
Clock input: This is absent when the Data Interface attribute's value is "One asynchronous load/store port." In other circumstances, when ld is 0, and this input rises from 0 to 1 (and sel is 1/undefined and clr is 0), then the value at the currently selected address changes to whatever value is at the D pin. As long as the clock input remains 0 or 1, though, the D value will not be stored into memory.
ld on south edge (input, bit width 1)
Load: Selects whether the RAM should emit (on D) the value at the current address (A). This output behavior is enabled if out is 1 or undefined; if out is 0, then no value is pushed onto D - but if there is a combined load/store port, stores will be enabled.
clr on south edge (input, bit width 1)
Clear: When this is 1, all values in memory are pinned to 0, no matter what the other inputs are.
Attributes
When the component is selected or being added, the digits '0' through '9' alter its Address Bit Width attribute and Alt-0 through Alt-9 alter its Data Bit Width attribute.
Address Bit Width
The bit width of the address bits. The number of values stored in RAM is 2addrBitWidth.
Data Bit Width
The bit width of each individual value in memory.
Data Interface
Configures which of the three interfaces are used for communicating data into and out of the component.
Poke Tool Behavior
See poking memory in the User's Guide.
Text Tool Behavior

None.
Menu Tool Behavior
See pop-up menus and files in the User's Guide.
Back to Library Reference

ROM

Library:
Memory
Introduced:
2.1.0
Appearance:

Behavior
The ROM component stores up to 16,777,216 values (specified in the Address Bit Width attribute), each of which can include up to to 32 bits (specified in the Data Bit Width attribute). A circuit can access the current values in ROM, but it cannot change them. The user can modify individual values interactively via the Poke Tool, or the user can modify the entire contents via the Menu Tool. Unlike the RAM component, the ROM component's current contents are stored as an attribute of the component. Thus, if a circuit containing a ROM component is used twice, then both ROM components will hold the same values. Also because of this behavior, the current ROM contents are stored in files created by Logisim.
Current values are displayed in the component. Addresses displayed are listed in gray to the left of the display area. Inside, each value is listed using hexadecimal. The value at the currently selected address will be displayed in inverse text (white on black).
Pins
A on west edge (input, bit width matches Address Bit Width attribute)
Selects which of the values are currently being accessed by the circuit.
D on east edge (input/output, bit width matches Data Bit Width attribute)
Outputs the value at the currently selected address at the D pin if sel is 1 or floating. If sel is 0, then D will be floating.
sel on south edge (input, bit width 1)
If you have just one ROM module, ignore this input. If you have multiple ROM modules in parallel, you can use this input to enable or disable the entire ROM module, based on whether the value is 1 or 0. In other words, when this is 0, no value is emitted on the D output.
Attributes
When the component is selected or being added, the digits '0' through '9' alter its Address Bit Width attribute and Alt-0 through Alt-9 alter its Data Bit Width attribute.
Address Bit Width
The bit width of the address bits. The number of values stored in ROM is 2addrBitWidth.
Data Bit Width
The bit width of each individual value in memory.
Contents
Stores the contents of memory.
Poke Tool Behavior
See poking memory in the User's Guide.
Text Tool Behavior
None.
Menu Tool Behavior
See pop-up menus and files in the User's Guide.
Back to Library Reference

Button

Library:
Input/Output
Introduced:
2.1.3
Appearance:

Behavior
Outputs 0 normally; but when the user is pressing the the button using the Poke Tool, the output is 1.
Pins
A button has only one pin, a 1-bit output, which is 0 except when the user is pressing the button using the Poke Tool, when it is 1.
Attributes
When the component is selected or being added, the arrow keys alter its Facing attribute.
Facing
The location of the output pin relative to the component.
Color
The color with which to display the button.
Label
The text within the label associated with the component.
Label Location
The location of the label relative to the component.
Label Font
The font with which to render the label.
Label Color
The color with which to draw the label.
Poke Tool Behavior
When the mouse button is pressed, the component's output will be 1. Upon releasing the mouse button, the output reverts back to 0.
Text Tool Behavior
Allows the label associated with the component to be edited.
Back to Library Reference

Joystick

Library:
Input/Output
Introduced:
2.2.0
Appearance:

Behavior
The user can drag the red knob within the rounded-square area, and the outputs update to indicate the knob's current x- and y-coordinates. This is meant to emulate the joysticks known from the days of classical arcade games.
Pins
West edge, north pin (output, bit width matches Bit Width attribute)
Indicates knob's x-coordinate, to be interpreted as an unsigned integer whose value will never be 0. Thus, a value of 1 represents the far left, and the maximum value for the bit width indicates the far right. When the knob is at rest (in the center), the value has the bit pattern 10...00.
West edge, south pin (output, bit width matches Bit Width attribute)
Indicates knob's y-coordinate, whose value ranges as with the x-coordinate pin. When the knob is pulled to the top, this output's value is 1, and when the knob is pulled to the bottom, the output is the maximum value for the bit width selected.
Attributes

When the component is selected or being added, Alt-2 through Alt-5 alter its Bit Width attribute.
Bit Width
The number of bits used to indicate each of the knob's coordinates.
Color
The knob's color as it is drawn on the screen.
Poke Tool Behavior
Pressing the mouse button while within the joystick area moves the knob to that location and updates the outputs. Dragging the mouse continues to move the knob and update the outputs, keeping the knob within the joystick's area. Releasing the mouse button reverts the knob back to its rest position.
Text Tool Behavior
None.
Back to Library Reference

Keyboard

Library:
Input/Output
Introduced:
2.2.0
Appearance:

Behavior
This component allows the circuit to read keys typed from the keyboard - as long as the keys are representable in the 7-bit ASCII code. After clicking the component using the poke tool, the user can type characters, which accumulate in a buffer. At all times, the ASCII value for the leftmost character in the buffer is sent out the rightmost output. When the clock input is triggered, the leftmost character disappears from the buffer and the new leftmost character is sent on the rightmost output.
The supported characters for the buffer include all the printable ASCII characters, as well as space, newline, backspace, and control-L. In addition, the left-arrow and right-arrow keys move the cursor within the buffer, and the delete key deletes the character to the right of the cursor (if any).
The component is asynchronous in the sense that when the buffer is empty and the user types a character, that character is sent immediately as an output, without any wait for a clock pulse.
Pins
West edge, marked by a triangle (input, bit width 1)
Clock - when triggered while the read-enable pin isn't 0, the leftmost character from the buffer is deleted, and the outputs are updated to reflect the buffer's new status.
South edge, leftmost pin (input, bit width 1)
Read Enable - when 1 (or floating or error), a clock edge will consume the leftmost character from the buffer. The clock input is ignored when Read Enable is 0.
South edge, second pin from left (input, bit width 1)
Clear - when 1, the buffer is emptied and does not accept further characters.
South edge, second pin from right (output, bit width 1)
Available - this is 1 when the buffer contains at least one character and 0 when the buffer is empty.
South edge, rightmost pin (output, bit width 7)
Data - the 7-bit ASCII code for the leftmost character in the buffer, or 0 if the buffer is empty.
Attributes
Buffer Length
The number of characters that the buffer can hold at once.
Trigger
If the value is Rising Edge, then when the clock input changes from 0 to 1, the leftmost character is consumed (when enabled by the Read Enable input). If it is Falling Edge, , then this happens when the clock input changes from 1 to 0.
Poke Tool Behavior

Pressing the mouse button into the component gives keyboard focus to the component, and a vertical-bar cursor will be displayed.
Each character typed will then be inserted into the buffer, as long as the buffer hasn't reached its capacity and the character is one of those that the component supports: the printable characters within the 7-bit ASCII code, as well as space, backspace, newline, and control-L. Additionally, the user may type the left-arrow and right-arrow keys to change the location of the cursor within the buffer, and the user may type the delete key to delete the buffer character (if any) just to the right of the cursor.
Text Tool Behavior
None.
Back to Library Reference

LED

Library:
Input/Output
Introduced:
2.1.3
Appearance:

Behavior
Displays the value of its input by coloring the LED (as specified by its Color attribute) or not depending on whether the input is 1 or 0.
(The LED component is basically redundant with an output pin, except for a somewhat different appearance. Some users, though, thought it would be nice to include.)
Pins
A LED has only one pin, a 1-bit input which is used to determine whether to display the LED colored (when the input is 1) or darkened (when the input is anything else).
Attributes
When the component is selected or being added, the arrow keys alter its Facing attribute.
Facing
The location of the input pin relative to the component.
Color
The color to display when the input value is 1.
Active On High?
If yes , then the LED is colored when the input is 1. If no , it is colored when the input is 0.
Label
The text within the label associated with the component.
Label Location
The location of the label relative to the component.
Label Font
The font with which to render the label.
Label Color
The color with which to draw the label.
Poke Tool Behavior
None.
Text Tool Behavior
Allows the label associated with the component to be edited.
Back to Library Reference

7-Segment Display

Library:
Input/Output

Introduced:
2.1.3
Appearance:

Behavior
Displays the values of its eight one-bit inputs. Segments are either colored or light gray depending on the inputs. The correspondence is as follows.

(Manufacturers vary as to how they map inputs to segments; the correspondence used here is based on Texas Instruments' TIL321.)
Pins
North edge, first from left (input, bit width 1)
Controls the middle horizontal segment.
North edge, second from left (input, bit width 1)
Controls the upper vertical segment on the left side.
North edge, third from left (input, bit width 1)
Controls the upper horizontal segment.
North edge, fourth from left (input, bit width 1)
Controls the upper vertical segment on the right side.
South edge, first from left (input, bit width 1)
Controls the lower vertical segment on the left side.
South edge, second from left (input, bit width 1)
Controls the bottom horizontal segment.
South edge, third from left (input, bit width 1)
Controls the lower vertical segment on the right side.
South edge, fourth from left (input, bit width 1)
Controls the decimal point.
Attributes
On Color
The color with which to draw the display segments and decimal point when they are on.
Off Color
The color with which to draw the display segments and decimal point when they are off.
Background
The color with which to draw the display's background (transparent by default).
Active On High?
If yes , then the segments light when the corresponding input is 1. If no , they light when the corresponding input is 0.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Hex Digit Display

Library:
Input/Output
Introduced:
2.2.0
Appearance:

Behavior
Using a seven-segment display, shows the hexadecimal digit corresponding to the four-bit input. If any of the inputs are not 0/1 (either floating or error), then the display shows a dash ('-'). A separate one-bit input controls the display of the decimal point.

Pins
South edge, first from left (input, bit width 4)
This input is interpreted as an unsigned four-bit number, and the corresponding hexadecimal digit is displayed. If any of the bits are floating or error, then a dash ('-') is displayed.
South edge, second from left (input, bit width 1)
Controls the decimal point. If this is left unconnected, the decimal point remains off.
Attributes
On Color
The color with which to draw the display segments and decimal point when they are on.
Off Color
The color with which to draw the display segments and decimal point when they are off.
Background
The color with which to draw the display's background (transparent by default).
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

LED Matrix

Library:
Input/Output
Introduced:
2.2.0
Appearance:

Behavior
Displays a small grid of pixels, whose values are determined by the current inputs. The grid can have up to 32 rows and 32 columns.
Pins
The interface to the component varies depending on the value of the Input Format attribute. It has three possible values.
Columns
The inputs are lined along the component's south edge, with one multibit input for each column of the matrix. Each input has as many bits as there are rows in the matrix, with the low-order bit corresponding to the southmost pixel in the column. A 1 indicates to light the corresponding pixel, while a 0 indicates to keep the pixel dim. If any of the bits for a column are either floating or error values, then all pixels in the column are lit.
Rows
The inputs are lined along the component's west edge, with one multibit input for each row of the matrix. Each input has as many bits as there are columns in the matrix, with the low-order bit corresponding to the rightmost pixel in the row. As with the Columns format, a 1 indicates to light the corresponding pixel, and a 0 indicates to keep the pixel dim. If any bits for a row are floating or error values, then all pixels in the row are lit.
Select Rows/Columns
There are two inputs on the component's west edge. The upper multibit input has as many bits as there are columns in the matrix, with the low-order bit corresponding to the rightmost column. The lower multibit input has as many bits as there are rows in the matrix, with the low-order bit corresponding to the bottom row. If any bits in either input are floating or error values, all pixels in the matrix are lit. Normally, though, a pixel at a particular row-column location is lit if the corresponding column bit in the upper input is 1 and the corresponding row bit in the lower input is 1. For example, for a 5x7 matrix, if the first input is 01010 and the second is 0111010, then the second and fourth columns are lit for the second, third, fourth, and sixth rows; the result appears to

be a pair of exclamation points. (This input format may seem unintuitive, but LED matrixes are sold commercially with exactly this interface. Lite-On sells such components, for example.)
Attributes
Input Format (read-only after component is created)
Selects how the pins correspond to pixels, as outlined above.
Matrix Columns
Selects how many columns are in the matrix, which may range from 1 up to 32.
Matrix Rows
Selects how many rows are in the matrix, which may range from 1 up to 32.
On Color
Selects the color of a pixel when it is lit.
Off Color
Selects the color of a pixel when it is dim.
Light Persistence
When this is other than 0, a pixel that is lit remains lit for the given number of clock ticks after the component's inputs indicate that the pixel should become dim.
Dot Shape
The square option means that each pixel is drawn as a 10x10 square, filling the component with no gaps between pixels. The circle option means that each pixel is drawn as a diameter-8 circle, with gaps between each circle. The circle option is more difficult to interpret, but it more closely approximates the off-the-shelf LED matrix components.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

TTY

Library:
Input/Output
Introduced:
2.2.0
Appearance:

Behavior
This component implements a very simple dumb terminal. It receives a sequence of ASCII codes and displays each printable character. When the current row becomes full, the cursor moves to the following line, possibly scrolling all current rows up if the cursor was already in the bottom row. The only supported control sequences are: backspace (ASCII 8), which deletes the last character in the final row, unless the final row is already empty; newline (ASCII 10), which moves the cursor to the beginning of the following line, scrolling if necessary; and form-feed (ASCII 12, typed as control-L), which clears the screen.
Pins
West edge, upper pin (input, bit width 7)
Data - this is the ASCII value of the next character to be entered into the terminal.
West edge, lower pin marked by triangle (input, bit width 1)
Clock - when triggered while the write-enable pin isn't 0, the current ASCII value on the Data input is processed by the terminal.
South edge, leftmost pin (input, bit width 1)
Write Enable - when 1 (or floating or error), a clock edge will result in processing a new character from the data input. The clock and data inputs are ignored when Write Enable is 0.
South edge, second pin from left (input, bit width 1)
Clear - when 1, the terminal is cleared of all data, and all other inputs are ignored.
Attributes

Rows
The number of rows displayed in the terminal.
Columns
The maximum number of characters displayed in each row of terminal.
Trigger
If the value is Rising Edge, then when the clock input changes from 0 to 1, the data input is processed (when enabled by the write-enable and clear inputs). If it is Falling Edge, , then this happens when the clock input changes from 1 to 0.
Color
The color with which to draw the text appearing in the terminal.
Background
The color with which to draw the terminal's background.
Poke Tool Behavior
None.
Text Tool Behavior
None.
Back to Library Reference

Poke Tool

Library:
Base
Introduced:
2.0 Beta 1
Behavior
The Poke Tool is for manipulating the current values associated with components. The precise behavior of the Poke Tool varies depending on which component is clicked; this behavior is documented in the `Poke Tool Behavior' section of each individual component. The following components all have support for the Poke Tool.
Base library
Pin Clock
Memory library
D/T/J-K/S-R Flip-Flop Register Counter Shift Register RAM ROM
Input/Output library
Button Joystick Keyboard
Also, clicking a wire segment using the Poke tool displays the value currently carried by the wire, as described on the Wiring Tool's page.
Attributes
None. Clicking on a component supporting the Poke Tool, though, will display that component's attributes.
Back to Library Reference

Edit Tool

Library:
Base
Introduced:
2.3.0
Behavior
The Edit tool allows the user to rearrange existing components and to add wires. Exactly what the tool does depends on where the user presses the mouse on the canvas.
When the mouse is over a wiring point for an existing component, or if it is atop a current wire, the Edit Tool will display a small green circle around the mouse's location. Pressing the button there initiates the addition of a new wire. But if the user doesn't drag the mouse far enough to initiate a

wire before releasing the button, the press is treated as a mouse click, and so the wire is simply added into the current selection.
The bit width of an added wire is inferred from the components to which it is connected. If it is not attached to any components, the wire will be drawn gray to indicate that its bit width is unknown; if the components at the locations that the wire helps to connect disagree on the bit width, then the wire will be drawn orange to indicate the conflict, and the wire will in fact refuse to carry any values at all until the user resolves the conflict.
All wires in Logisim are either horizontal or vertical, never diagonal.
Wires are non-directional; that is, they carry values from either endpoint to the other. Indeed, a wire can carry values in both directions simultaneously: In the below example, a bit flows from the upper input at left through the center wire, then it circles back through the center wire, and then it circles forward again through the center wire before reaching the output at lower right.

A single drag of the mouse can create multiple wire segments. The precise process is a little confusing in its description; but it works quite intuitively in practice: If you request a particular wire segment using the Wiring Tool, that segment will be split apart wherever it hits a pin for an existing component, or wherever it hits the endpoint of an existing wire segment. Also, if an endpoint of any of the new wire segments hit somewhere in the middle of an existing wire, then that wire will be split into multiple segments itself.
You can also shorten or delete an existing wire segment by initiating a drag at the terminus of the segment and then drawing backwards across the segment. During the drag, the shortening is indicated by drawing a white line over of the portion of the wire that will be removed.
Some components draw short stubs to which wires can connect, such as the OR gate and controlled buffer. Logisim will silently correct attempts to create wires that slightly overshoot the stub's end.
If, however, the user presses the Alt key at a point in the middle of the wire, then the green circle will disappear. A mouse press selects the wire, and a mouse drag moves it.
Pressing the mouse button while it is within a currently selected component begins a drag moving all elements of the selection.
By default, Logisim will compute a way to add new wires so that no existing connections are lost during the move. (Sometimes it will delete or shorten existing wires.) If you're performing a move where you do not want these changes to be made, you can press the shift key during the move. If you want to disable this behavior entirely, go to Project > Options, select the Canvas tab, and uncheck the Keep Connections When Moving box; in this case, the connections are computed only when the shift key is down.
Dragging a selection can lead to unexpected behavior from wires: If you drag a selection including some wires on top of some other wires, all wires are merged, and the merged wires are placed into the selection. As a result, if you drag the selection a second time, the wires previously at the location will not be left behind. This behavior is necessary to keep with the intuitive behavior of wires in Logisim, where wires never overlap. And it does not normally constitute a major problem: Logisim will draw the full selection in the midst of dropping, and you should not drop it until you are sure it is in the correct location.
Pressing the mouse within an unselected component (but not at one of the component's wiring points) drops all components from the current selection and selects instead the component(s) containing the clicked location.
Shift-clicking the mouse within a component toggles that component's presence within the selection. If multiple components include the same location, all components' presence will be toggled.
Dragging the mouse starting at a location not contained within any components drops all components from the current selection and initiates a rectangular selection. All component(s) contained by the rectangle will be placed into the selection.
Shift-dragging the mouse starting at a location not contained within any components initiates a rectangular selection. The presence in the selection of all component(s) contained by the rectangle will be toggled.
However, if the Alt key is pressed at a location not contained within any components, this initiates the addition of a new wire. A small green circle is drawn in such a circumstance to indicate this.

After selecting the desired items in the selection, you can of course cut/copy/paste/delete/duplicate all the items via the Edit menu.
Some keys have an effect with the Edit Tool.
The arrow keys change the Facing attribute for all components in the selection that have such an attribute.
The Delete and Backspace keys will delete everything in the selection from the circuit.
The Insert and MenuKey-D keys will create a duplicate of the currently selected components.
Logisim's behavior when duplicating a selection or pasting the clipboard into a circuit is somewhat peculiar: It will not immediately place the components into the circuit; instead, the selection will be a collection of "ghosts," which will be dropped into the circuit as soon as they are either dragged to another location or removed from the selection. (This peculiar behavior is necessary because pasting will otherwise merge the wires of the selection into the current circuit at once, and the wires there previously will be dragged with the pasted clipboard if the user wants to move the pasted components somewhere else.)
Attributes
None. Selecting a component, though, will display its attributes. With multiple components selected, attributes shared by all are shown, blank if they have different values and otherwise with the value they all have in common. (Wires are ignored if there are any non-wires in the selection.) Changes to the attribute value affect all selected components.
Back to Library Reference

Select Tool

Library:
Base
Introduced:
2.0 Beta 1
Behavior
Allows individual components to be placed into the current selection. There are a number of actions possible with this tool.
Pressing the mouse button while it is within a currently selected component begins a drag moving all components of the selection.
By default, Logisim will compute a way to add new wires so that no existing connections are lost during the move. (Sometimes it will delete or shorten existing wires.) If you're performing a move where you do not want these changes to be made, you can press the shift key during the move. If you want to disable this behavior entirely, go to Project > Options, select the Canvas tab, and uncheck the Keep Connections When Moving box; in this case, the connections are computed only when the shift key is down.
Dragging a selection can lead to unexpected behavior from wires: If you drag a selection including some wires on top of some other wires, all wires are merged, and the merged wires are placed into the selection. As a result, if you drag the selection a second time, the wires previously at the location will not be left behind. This behavior is necessary to keep with the expected behavior of wires in Logisim. And it does not normally constitute a major problem: Logisim will draw the full selection in the midst of dropping, and you should not drop it until you are sure it is in the correct location.
Otherwise, clicking the mouse within a component drops all components from the current selection and selects instead the component(s) containing the clicked location.
Shift-clicking the mouse within a component toggles that component's presence within the selection. If multiple components include the same location, all components' presence will be toggled. None of this will happen, though, if shift-clicking is mapped to another tool instead (via the project options window's Mouse tab).
Dragging the mouse starting at a location not contained within any components drops all components from the current selection and initiates a rectangular selection. All component(s) contained by the rectangle will be placed into the selection.

Shift-dragging the mouse starting at a location not contained within any components initiates a rectangular selection. The presence in the selection of all component(s) contained by the rectangle will be toggled. This will not happen, though, if shift-clicking is mapped to another tool instead. After selecting the desired items in the selection, you can of course cut/copy/paste/delete all the items via the Edit menu.
Logisim's behavior when pasting the clipboard into a circuit is somewhat peculiar: It will not immediately place the components into the circuit; instead, the selection will be a collection of "ghosts," which will be dropped into the circuit as soon as they are either dragged to another location or removed from the selection. (This peculiar behavior is necessary because pasting will otherwise merge the wires of the selection into the current circuit at once, and the wires there previously will be dragged with the pasted clipboard if the user wants to move the pasted components somewhere else.)
Attributes
None. Selecting a component, though, will display its attributes. With multiple components selected, attributes shared by all are shown, blank if they have different values and otherwise with the value they all have in common. (Wires are ignored if there are any non-wires in the selection.) Changes to the attribute value affect all selected components.
Back to Library Reference

Wiring Tool

Library:
Base
Introduced:
2.0 Beta 1
Behavior
The wiring tool is the tool for creating wire segments that carry values from one endpoint to another. The bit width of these values can be anything; exactly which bit width is automatically inferred from the components to which the wires are ultimately attached. If it is not attached to any components, the wire will be drawn gray to indicate that its bit width is unknown; if the components at the locations that the wire helps to connect disagree on the bit width, then the wire will be drawn orange to indicate the conflict, and the wire will in fact refuse to carry any values at all until the user resolves the conflict.
A single drag of the mouse can create multiple wire segments. The precise process is a little confusing in its description; but it works quite intuitively in practice: If you request a particular wire segment using the Wiring Tool, that segment will be split apart wherever it hits a pin for an existing component, or wherever it hits the endpoint of an existing wire segment. Also, if an endpoint of any of the new wire segments hit somewhere in the middle of an existing wire, then that wire will be split into multiple segments itself.
For some components that draw short stubs to which wires can connect (such as an OR gate or a controlled buffer), Logisim will silently correct attempts to create wires that slightly overshoot the stub's end.
You can also shorten an existing wire segment using the Wiring Tool, using a drag that starts or ends at a terminus of the segment, and that overlaps the existing segment.
All wires in Logisim are either horizontal or vertical.
Wires are also non-directional; that is, they carry values from either endpoint to the other. Indeed, a wire can carry values in both directions simultaneously; the center wire in the below example is doing this.

Attributes
The wiring tool does not itself have attributes, but the wires that it creates do.
Direction
Indicates whether the wire is horizontal or vertical. The value of this attribute cannot be changed.
Length
Indicates how many pixels long the wire is. The value of this attribute cannot be changed.

Poke Tool Behavior
When you click an existing wire segment using the Poke Tool, Logisim displays the current value traveling through that wire. The behavior is particularly useful for multi-bit wires, whose black color provide no visual feedback about what value the wire is carrying.
For multi-bit values, you can configure exactly how the value is displayed (in binary, decimal, or hexadecimal, for example) using the Layout pane of the Logisim Preferences dialog box.
Back to Library Reference

Text Tool

Library:
Base
Introduced:
2.0 Beta 1
Behavior
The text tool allows you to create and edit labels associated with components. Which components support labels are indicated in the 'Text Tool Behavior' section of their documentation. As of the current release, the following components in the built-in libraries support labels.
Base library
Pin Clock Label Probe
Memory library
D/T/JK/SR Flip-Flop Register Counter Shift Register Random
Input/Output library
Button LED
For components that can take a label but have none assigned to it currently, you can click anywhere within the component to add a label. If there is already a label, you need to click within the label. If you click at a point where there is not currently a label to be edited, Logisim will initiate the addition of a new Label component.
In the current version of Logisim, text editing features are still fairly primitive. Selections of a region of text within a label is impossible. There is no way to insert a line break into a label.
Attributes
The attributes for the tool are the same as for the label component. These attributes have no effect when editing the label on an existing component, but they are imparted to any labels created using the text tool.
Clicking on a component supporting the Text Tool will display that component's attributes.
Back to Library Reference

Menu Tool

Library:
Base
Introduced:
2.0 Beta 1
Behavior
The menu tool permits the user to pull up a pop-up menu for components that already exist. By default, right-clicking or control-clicking a component will bring up this pop-up menu; however, the Mouse tab of the project options allows a user to configure the mouse buttons to work differently.
The pop-up menu for most components has two items.
Delete: Removes the component from the circuit.
Show Attributes: Places the component's attributes into the window's attribute table, so that the attribute values can be viewed and changed.
For some components, however, the menu has additional items. Subcircuits (that is, instances of using one circuit as a "black box" within another) are one example of this: In addition to the above two items, the pop-up menu includes another item.

View XXX: Changes the circuit layout being viewed and edited to be the subcircuit's layout instead. The values seen in the layout will be part of the same hierarchy as those of the supercircuit. (See the `Debugging subcircuits' section of the User's Guide.)
Other components may extend the pop-up menu also. In the built-in libraries of the current version of Logisim, the only such components are RAM and ROM.
Attributes
None.
Back to Library Reference

Label

Library:
Base
Introduced:
2.0 Beta 1
Appearance:

Behavior
This is a simple text label that can be placed anywhere in the circuit. It does not interact with values traveling through the circuit in any way, except inasmuch as it will be visible when the circuit is drawn.
In contrast to all other components in the current built-in libraries, label components can be placed anywhere on the canvas; they do not snap to the grid.
Pins
None.
Attributes
Text
The text appearing in the label. This value can be edited in the attribute table or, using the text tool, on the canvas.
Font
The font to use when drawing the label.
Horizontal Alignment
The horizontal positioning technique for the text relative to the label's official location (where the mouse was clicked in creating the label). "Left" means that the text should be drawn so that its left edge is at the location; "right" means that the text should be drawn so that its right edge is at the location; and "center" means that the text should be drawn so that its center (horizontally) is at the location.
Vertical Alignment
The vertical positioning technique for the text relative to the label's official location (where the mouse was clicked in creating the label). "Base" means that the baseline should intersect the location; "Top" means that the text's top should intersect the location; "Bottom" means that the text's bottom should intersect the location; and "Center" means that the text should be centered (vertically) at the location.
The text's top and bottom is computed based on the font's standard ascent and descent values; thus, even if the actual text contains no tall letters (such as b) or descending letters (such as g), it is assumed to contain such letters for the purposes of vertical positioning.
Poke Tool Behavior
None.
Text Tool Behavior
Allows the text appearing within the label to be edited.
Back to Library Reference